

INTRODUCTION

INTRODUCTION TO
SOFTWARE ENGINEERING

Vu Son Lam

08/2019

1

References

- [1] Ian Sommerville, *Software Engineering*, 9th ed., Addison-Wasley, 2011.
- [2] Pankaj Jalote, *A Concise Introduction to Software Engineering*, Springer, 2008.

2

Contents

1. Professional software development
2. Softwares vs Computer programs
3. Kinds of software products
4. Software engineering
5. Fundamental activities in a software process
6. Types of application
7. Frequently asked questions about software
8. Software problem

3

Contents

1. **Professional software development**
2. Softwares vs Computer programs
3. Kinds of software products
4. Software engineering
5. Fundamental activities in a software process
6. Types of application
7. Frequently asked questions about software
8. Software problem

4

History of software engineering

- The notion of 'software engineering' was first proposed in **1968** at a conference held to discuss what was then called the 'software crisis'.



<https://medium.com/@ryancohane/has-the-software-crisis-passed-d45ce975a1e7>

5

History of software engineering

- Software crisis
 - Projects running over-budget
 - Projects running over-time
 - Software was very inefficient
 - Software was of low quality
 - Software often did not meet requirements
 - Projects were unmanageable and code difficult to maintain
 - Software was never delivered

Nguồn: https://en.wikipedia.org/wiki/Software_crisis

6

History of software engineering

- Throughout the 1970s and 1980s, a variety of new software engineering techniques and methods were developed, such as
 - structured programming,
 - information hiding and
 - object-oriented development.
- Tools and standard notations were developed and are now extensively used.

7

Professional software development

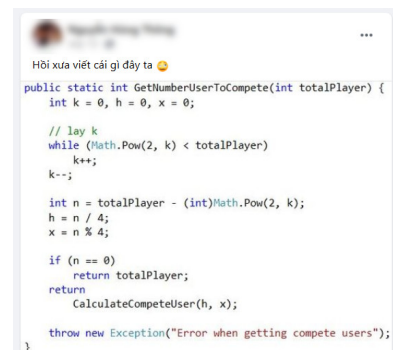
- Lots of people write programs:
 - People in business write spreadsheet programs to simplify their jobs,
 - Scientists and engineers write programs to process their experimental data, and
 - Hobbyists write programs for their own interest and enjoyment.
- **Professional software development:**
 - Software is developed for specific business purposes, for inclusion in other devices, or as software products such as information systems, etc.
 - Professional software, intended for use by someone apart from its developer, is usually developed by teams rather than individuals. It is maintained and changed throughout its life.

8

Professional software development

- One of the important differences between professional and amateur software development:
 - If you are writing a program for yourself, **no one else will use it** and you **don't have to worry about writing program guides**, documenting the program design, etc.
 - However, if you are writing software that other people will use and other engineers will change then you usually **have to provide additional information** as well as the code of the program.

9



```
***
Hỏi xúa viết cái gì đây ta 😊
public static int GetNumberUserToCompete(int totalPlayer) {
    int k = 0, h = 0, x = 0;

    // lay k
    while ((Math.Pow(2, k) < totalPlayer)
        k++;
    k--;

    int n = totalPlayer - (int)Math.Pow(2, k);
    h = n / 4;
    x = n % 4;

    if (n == 0)
        return totalPlayer;
    return
        CalculateCompeteUser(h, x);

    throw new Exception("Error when getting compete users");
}
```

10

Contents

1. Professional software development
2. **Softwares vs Computer programs**
3. Kinds of software products
4. Software engineering
5. Fundamental activities in a software process
6. Types of application
7. Frequently asked questions about software
8. Software problem

11

Softwares vs Computer programs

- A professionally developed software system is often more than a single program. The system usually consists of:
 - **A number of separate programs** and
 - **Configuration files** that are used to set up these programs.
 - **Documents:**
 - system documentation: describes the structure of the system,
 - user documentation, which explains how to use the system,
 - websites for users to download recent product information.

12

Software definition

• What is software?

- Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market. (Ilan2011)

13

Contents

1. Professional software development
2. Softwares vs Computer programs
3. **Kinds of software products**
4. Software engineering
5. Fundamental activities in a software process
6. Types of application
7. Frequently asked questions about software
8. Software problem

14

Kinds of software products

- **Generic products** These are stand-alone systems that are produced by a development organization and sold on the open market to any customer who is able to buy them.
- Examples of this type of product include software for PCs such as databases, word processors, drawing packages, and project-management tools:
 - MySQL, SQL Server, Oracle
 - Microsoft project, Redmi
 - Microsoft Offices: Word, Excel, Powerpoint.
 - Photoshop, Corel,...

15

Kinds of software products

- **Customized (or bespoke) products** These are systems that are commissioned by a particular customer. A software contractor develops the software especially for that customer.
- Examples of this type of software include control systems for electronic devices, systems written to support a particular business process, and air traffic control systems.
 - Library management system for Quy Nhon university.
 - BIDV smart banking
 - VN Railway ticket...

16

Kinds of software products

D What is difference between these types?

If you a software company, which software products you like to develop?
Customized products or Generic products.

17

Kinds of software products

- An important difference between these types of software is that:
 - In generic products, the organization that develops the software **controls the software specification**.
 - For custom products, the **specification** is usually developed and **controlled by the organization that is buying the software**. The software developers must work to that specification.

18

Kinds of software products

- However, the **distinction** between these system product types is becoming increasingly **blurred**.
 - More and more systems are now being **built with a generic product as a base**, which is **then adapted** to suit the requirements of a customer.
 - Enterprise Resource Planning (ERP) systems, such as the SAP system, are the best examples of this approach. Here, a large and complex system is adapted for a company by incorporating information about business rules and processes, reports required, and so on.
 - Quy Nhon University's Portal (customized from a base product)

19

Contents

1. Professional software development
2. Softwares vs Computer programs
3. Kinds of software products
- 4. **Software engineering**
5. Fundamental activities in a software process
6. Types of application
7. Frequently asked questions about software
8. Software problem

20

Software engineering

- Software engineering is an **engineering discipline** that is concerned with **all aspects of software production** from the early stages of system specification through to maintaining the system after it has gone into use.
- **Engineering discipline**
 - Engineers make things work. They apply **theories**, **methods**, and **tools** where these are appropriate. However, they use them selectively and always try to discover solutions to problems even when there are no applicable theories and methods.

21

Software engineering

- **All aspects of software production**
 - Software engineering is not just concerned with the **technical processes of software development**.
 - It also includes activities such as **software project management** and the **development of tools, methods**, and **theories** to support software production.

22

Why software engineering is important?

- Software engineering is important for two reasons:
 - More and more, individuals and society rely on advanced software systems. We need to be able to produce **reliable** and **trustworthy** systems **economically** and **quickly**.
 - It is **usually cheaper**, in the long run, to use software engineering methods and techniques for software systems rather than just write the programs as if it was a personal programming project. For most types of systems, the majority of costs are the costs of changing the software after it has gone into use.
- **Quality, Cost, Schedule.**

23

Contents

1. Professional software development
2. Softwares vs Computer programs
3. Kinds of software products
4. Software engineering
- 5. **Fundamental activities in a software process**
6. Types of application
7. Frequently asked questions about software
8. Software problem

24

Fundamental activities in a software process

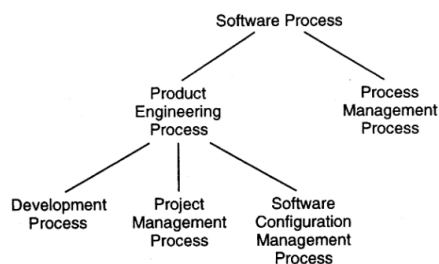
- The systematic approach that is used in software engineering is sometimes called a software process.
- **A software process is a sequence of activities that leads to the production of a software product.**
- There are four fundamental activities that are common to all software processes.

25

Fundamental activities in a software process

- **Software specification**, where customers and engineers define the software that is to be produced and the constraints on its operation.
- **Software development**, where the software is designed and programmed.
- **Software validation**, where the software is checked to ensure that it is what the customer requires.
- **Software evolution**, where the software is modified to reflect changing customer and market requirements.

26



27

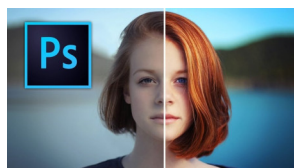
Contents

1. Professional software development
2. Softwares vs Computer programs
3. Kinds of software products
4. Software engineering
5. Fundamental activities in a software process
6. **Types of application**
7. Frequently asked questions about software
8. Software problem

28

Types of application

- **Stand-alone applications** These are application systems that run on a local computer, such as a PC. They include all necessary functionality and do not need to be connected to a network.
 - Examples of such applications are office applications on a PC, CAD programs, photo manipulation software, etc.



Types of application

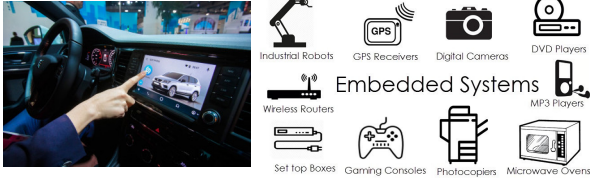
- **Interactive transaction-based applications** These are applications that execute on a remote computer and that are accessed by users from their own PCs or terminals.
 - Includes web application such as e-commerce applications where you can interact with a remote system to buy goods and services.
 - program and cloud-based services, such as mail and photo sharing.
 - Interactive applications often incorporate a large data store that is accessed and updated in each transaction.



30

Types of application

- **Embedded control systems:** control and manage hardware devices.
 - software that controls anti-lock braking in a car, and
 - software in a microwave oven to control the cooking process.



31

Types of application

- **Batch processing systems** These are business systems that are designed to **process data in large batches**. They process large numbers of individual inputs to create corresponding outputs.
 - Examples of batch systems include periodic billing systems, such as **phone billing systems**, and **salary payment systems**.

32

Types of application

- **Entertainment systems** These are systems that are primarily for personal use and which are intended to entertain the user.
 - Games...



33

Types of application

- **Systems for modeling and simulation:** developed by scientists and engineers to model physical processes or situations.



Gennymotion



34

Types of application



VISSIM là phần mềm mô phỏng, phân tích luồng giao thông, hỗ trợ công tác phân luồng, góp phần giảm tắc nghẽn giao thông trong các đô thị. Được áp dụng ở một số thành phố: Hà Nội, Huế, Đà Nẵng...

Nguồn: <https://www.ptvgroup.com/en/solutions/products/ptv-vissim/>
<https://www.cic.com.vn/san-pham/phan-mem-mo-phong-giao-thong-ptv-vissim>

35

Types of application

- **Data collection systems** These are systems that collect data from their environment using a set of sensors and send that data to other systems for processing.



36

Types of application

- **Systems of systems:** These are systems that are composed of a number of other software systems.



37

Choosing software engineering techniques

- We use different software engineering techniques for each type of system because the software has quite different characteristics.
- Some approaches for software development:
 - Plan-drive approach
 - Agile approach



Which approach we should use for:

- Embedded systems
- Web-based systems
- Systems of systems

38

Choosing software engineering techniques

- **For embedded systems:**
 - An embedded control system in an automobile is **safety-critical** and is burned into ROM when installed in the vehicle.
 - It is therefore **very expensive to change**. Such a system needs very **extensive verification** and **validation** so that the chances of having to recall cars after sale to fix software problems are minimized.
 - User interaction is minimal (or perhaps nonexistent) so there is no need to use a development process that relies on user interface prototyping.

39

Choosing software engineering techniques

- **For a web-based system**
 - An approach based on **iterative development and delivery** may be appropriate.
- **For system of systems:**
 - However, iterative development and delivery may be impractical for a system of systems, where **detailed specifications of the system interactions** have to be specified in advance so that each system can be separately developed.

40

Contents

1. Professional software development
2. Softwares vs Computer programs
3. Kinds of software products
4. Software engineering
5. Fundamental activities in a software process
6. Types of application
7. **Frequently asked questions about software**
8. Software problem

41

Frequently asked questions about software

- **What is software?**
 - Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market. (Ian 2011)
- **What is software engineering?**
 - Software engineering is an engineering discipline that is concerned with all aspects of software production.

42

Frequently asked questions about software

- **What are the fundamental software engineering activities?**
 - Software specification, software development, software validation, and software evolution.
- **What is the difference between software engineering and computer science?**
 - Computer science focuses on theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.

Nguồn: Ian 2011

43

Frequently asked questions about software

- **What are the costs of software engineering?**
 - Roughly **60% of software costs are development costs; 40% are testing costs.**
 - For custom software, evolution costs often exceed development costs.
- **What are the best software engineering techniques and methods?**
 - While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system.
 - For example, games should always be developed using a series of prototypes whereas safety critical control systems require a complete and analyzable specification to be developed.
 - You can't, therefore, say that one method is better than another

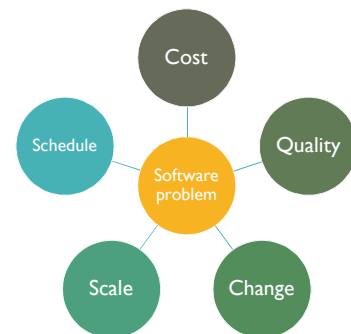
44

Contents

1. Professional software development
2. Softwares vs Computer programs
3. Kinds of software products
4. Software engineering
5. Fundamental activities in a software process
6. Types of application
7. Frequently asked questions about software
8. Software problem

45

Software problem



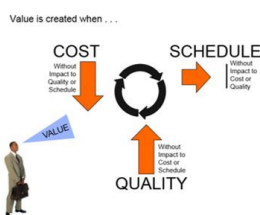
The major factors drive software engineering

46

Cost, schedule and quality

- The software should be produced
 - at reasonable cost,
 - in a reasonable time, and
 - should be of good quality

Các yêu cầu của một sản phẩm phần mềm



47

Cost

- Lines of code (LOC) or thousands of lines of code (KLOC) delivered is by far the most commonly used measure of software size in the industry.
- **The cost of developing software** is generally measured in terms of person-months of effort spent in development.
- And **productivity** is frequently measured in the industry in terms of **LOC** (or **KLOC**) per person-month.

48

Cost

- Software companies often charge the client for whom they are developing the software between **\$3,000 - \$15,000** per person-month [2].
- With a **productivity of 1000 LOC** per person-month, it means that each line of delivered code costs between **\$3 and \$15!**
- Assume a software has **50,000 LOC**
→ Cost **\$150,000 and \$750,000!**

49

Schedule

- Schedule is another important factor in many projects.
 - **Business trends** are dictating that the **time to market** of a product **should be reduced**; that is, the cycle time from concept to delivery should be small.
 - For software this means that it needs to be **developed faster**, and **within the specified time**.
 - Unfortunately, the history of software is full of cases where projects have been substantially late.

50

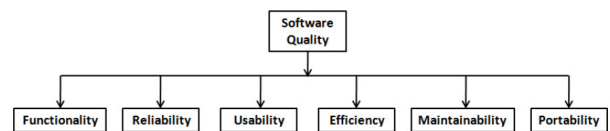
Productivity → Cost, Schedule

- Reducing the cost and the cycle time for software development are central goals of software engineering.
- Productivity in terms of output (KLOC) per person-month can adequately capture both cost and schedule concerns
 - Pursuit of **higher productivity** is a basic driving force behind software engineering and a **major reason** for **using the different tools and techniques**.

51

Quality

- Besides cost and schedule, the other major factor driving software engineering is quality.
- The international standard on software product quality* suggests that software quality comprises six main attributes:



* International Standards Organization. Software engineering — product quality. part 1: Quality model. Technical Report ISO9126-1, 2001.

52

Quality

- **Functionality.** The capability to provide functions which meet stated and implied needs when the software is used.
- **Reliability.** The capability to provide failure-free service.
- **Usability.** The capability to be understood, learned, and used.
- **Efficiency.** The capability to provide appropriate performance relative to the amount of resources used such as memory and processor cycles.
- **Maintainability.** The capability to be modified for purposes of making corrections, improvements, or adaptation.
- **Portability.** The capability to be adapted for different specified environments without applying actions or means other than those provided for this purpose in the product.

53

Quality

- With multiple dimensions to quality, **different projects may emphasize different attributes**, and a global single number for quality is not possible.
- However, **reliability is generally accepted to be the main quality criterion**. As unreliability of software is due to the presence of defects in the software, one measure of quality is the number of defects in the delivered software per unit size (generally taken to be thousands of lines of code, or KLOC).
- With this as the major quality criterion, **the quality objective is to reduce the number of defects per KLOC as much as possible**.
- Current best practices in software engineering have been able to reduce the defect density to **less than 1 defect per KLOC**.

54

Quality – What is a software defect?

- A Software **DEFECT** / **BUG** is a condition in a software product which
 - does not meet a software requirement (as stated in the requirement specifications) or
 - does not meet end-user expectation (which may not be specified but is reasonable).
- In other words, a defect is an error in coding or logic that causes a program to malfunction or to produce incorrect/unexpected results.

<http://softwaretestingfundamentals.com/defect/>

55

Scale and Change

- Though cost, schedule, and quality are the main driving forces for a project in our problem domain (of industry strength software), there are some other characteristics of the problem domain that also influence the solution approaches employed
 - Scale
 - Change

56

Scale

- Most industrial-strength software systems tend to be large and complex, requiring tens of thousands of lines of code.

Table 1.1: Size in KLOC of some well-known products.

Size (KLOC)	Software	Languages
980	gcc	ansic, cpp, yacc
320	perl	perl, ansic, sh
200	openssl	ansic, cpp, perl
100	apache	ansic, sh
65	sendmail	ansic
30,000	Red Hat Linux	ansic, cpp
40,000	Windows XP	ansic, cpp

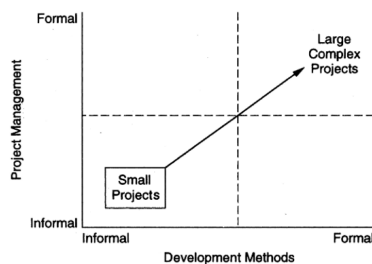
57

Scale

- Development of a large system requires a different set of methods compared to developing a small system.
 - Consider the problem of counting people in a room versus taking a census of a country.
- In small projects, informal methods for development and management can be used. However, for large projects, both have to be much more rigorous.

58

To successfully execute a project, a **proper method for engineering the system** has to be employed and the project has to be **tightly managed** to make sure that cost, schedule, and quality are under control.



59

Change



When and why changes occur in software development life cycle?

60

Change

- Change is another characteristic of the problem domain which the approaches for development must handle.
 - as the complete set of requirements for the system is generally not known (often cannot be known at the start of the project) or stated
 - as development proceeds and time passes, additional requirements are identified, which need to be incorporated in the software being developed.
- This need for changes requires that **methods for development embrace change and accommodate it efficiently.**
- Change requests can be quite disruptive to a project, and if not handled properly, can consume up to 30 to 40% of the development cost.

61

Change

- Though traditionally changes in software during maintenance have been distinguished from changes that occur while the development is taking place, these lines are blurring, as fundamentally the changes in both of these scenarios are similar
 - existing source code needs to be changed due to some changes in the requirements or due to some defects that need to be removed.

62

Change

- Overall, as the world changes faster, software has to change faster, even while under development.
- Changes in requirements are therefore a characteristic of the problem domain.
- In today's world, approaches that cannot accept and accommodate change are of little use—they can solve only those few problems that are change resistant.

63

Exercises / Discussion

1

What is the most important difference between **generic software product development** and **custom software development**?

What might this mean in practice for users of generic software products?

64

Exercises / Discussion

2

What are the **important attributes** that all **professional software** should have?

65

Exercises / Discussion

3

Explain, with examples, why **different application types** require **specialized software engineering techniques** to support their design and development?

66

Exercises 1

- The essential difference is that
 - In generic software product development, the specification is owned by the product developer.
 - For custom product development, the specification is owned and controlled by the customer.
- The implications of this are significant
 - The developer can **quickly decide to change the specification** in response to some external change (e.g. a competing product) but,
 - When the customer owns the specification, changes **have to be negotiated** between the customer and the developer and may have **contractual** implications.

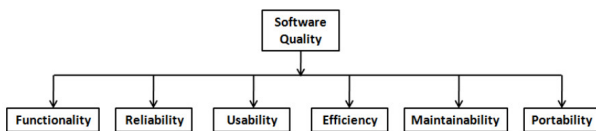
67

Exercises 1

- For users of generic products, this means
 - They have no control over the software specification so **cannot control the evolution** of the product.
 - The developer may decide to include/exclude features and change the user interface.
 - This could have implications for the **user's business processes** and add **extra training costs** when new versions of the system are installed.
 - It also may limit the **customer's flexibility to change** their own business processes.

68

Exercises 2



International Standards Organization. Software engineering — product quality. part 1: Quality model. Technical Report ISO9126-1, 2001.

69

Exercises 2

- **Functionality.** The capability to provide functions which meet stated and implied needs when the software is used.
- **Reliability.** The capability to provide failure-free service.
- **Usability.** The capability to be understood, learned, and used.
- **Efficiency.** The capability to provide appropriate performance relative to the amount of resources used such as memory and processor cycles.
- **Maintainability.** The capability to be modified for purposes of making corrections, improvements, or adaptation.
- **Portability.** The capability to be adapted for different specified environments without applying actions or means other than those provided for this purpose in the product.

70

Exercises 3

- Different application types require the use of different development techniques for a number of reasons:
 - Costs and frequency of change.
 - The software lifetime and delivery schedule.
 - The most important 'non-functional' requirements.

71

Exercises 3

- **Costs and frequency of change:**
 - Some systems (such as embedded systems in consumer devices) are **extremely expensive to change**; others, **must change frequently** in response to changing requirements (e.g. business systems).
 - Systems which are very expensive to change need **extensive upfront analysis** to ensure that the requirements are **consistent** and extensive validation to ensure that the system **meets its specification**. This is not cost-effective for systems that change very rapidly.

72

Exercises 3

- **The software lifetime and delivery schedule:**
 - Some software systems have a **relatively short lifetime** (many web-based systems), others have a **lifetime of tens of years** (large command and control systems). Some systems have to be **delivered quickly** if they are to be useful.
 - The techniques used to develop short-lifetime, rapid delivery systems (e.g. use of scripting languages, **prototyping**, etc.) are **inappropriate** for long-lifetime systems which require techniques that allow for long-term support such as **design modelling**.

73

Exercises 3

- **The most important 'non-functional' requirements:**
 - Different systems have **different priorities** for **non-functional requirements**.
 - For example, a real-time control system in an aircraft has **safety** as its principal priority; an interactive game has **responsiveness** and **usability** as its priority.
 - The techniques used to achieve safety are not required for interactive gaming; the **extensive UI design** required for games is not needed in safety-critical control systems.

74

Exercises 3



Nguồn ảnh: <https://www.engadget.com/2013/02/14/uk-thales-passive-radar/>

75

Exercises 3



Nguồn ảnh: <https://wccftech.com/hardcore-game-dev-bundle/>

76